

MISSIONE SPAZIALE: PULISCI LA GALASSIA

Istruzioni e spiegazione del codice — passo per passo, dall'inizio

Progetto Unity (C#) • Esame: Sistemi per la Riabilitazione e la Terapia Assistita
Autore: Filippo Cicirelli • Docente: prof. Vitantonio Bevilacqua

1. A cosa serve questo documento

Questo documento spiega **tutto quello che è stato costruito nel codice del gioco**, dall'inizio fino alla versione attuale. È scritto come una guida: si parte dall'idea, si vede com'è organizzato il progetto e poi si spiega, file per file, cosa fa ogni pezzo di codice e perché è stato scritto in quel modo.

Idea del gioco. Si guida un alieno chiamato *Astro* con il mouse. Astro deve raccogliere tutte le caramelle spaziali, poi prendere una chiave ed infine portarla alla porta per completare la missione. Lungo la strada bisogna evitare asteroidi e bombe. Ci sono 3 livelli con difficoltà crescente.

Scopo didattico (riabilitazione). Il gioco è pensato come esercizio di *coordinazione occhio-mano*: il movimento del mouse deve essere preciso ma non frustrante. Per questo molti aiuti sono regolabili (raggi di presa generosi, magnetismo della chiave, periodo "PRONTI", vite e tempo) così la difficoltà può essere adattata al paziente.

2. Come è organizzato il progetto

Tutto il codice del gioco si trova nella cartella `Assets/Codice/`. Una cosa importante da capire subito: **non ci sono immagini esterne e quasi nessuna configurazione fatta a mano nell'editor di Unity**. Tutto (telecamera, oggetti, disegni, interfaccia) viene creato *dal codice* quando si preme Play. Questo rende il progetto facile da leggere e da correggere: per cambiare qualcosa si modifica il codice, non si va a cercare nei menu di Unity.

File	Responsabilità in una frase
<code>Impostazioni.cs</code>	Tutti i numeri del gioco (vite, tempo, raggi, punti) in un posto solo.
<code>Avvio.cs</code>	Parte da solo al Play: crea telecamera, Gestore e Interfaccia.
<code>GestoreGioco.cs</code>	Il "cervello": punteggio, vite, tempo, livelli, vittoria e sconfitta.
<code>DefinizioneLivelli.cs</code>	Descrive i 3 livelli (dove vanno caramelle, asteroidi, bombe).
<code>CaricatoreLivelli.cs</code>	Costruisce in scena gli oggetti di un livello a partire dalla sua descrizione.
<code>FabbricaImmagini.cs</code>	Disegna da codice tutti gli sprite (pixel art 16×16).
<code>Astro.cs</code>	Il personaggio del giocatore: segue il mouse e controlla i contatti.
<code>Caramella.cs / Chiave.cs / Porta.cs</code>	Gli oggetti-obiettivo da raccogliere o raggiungere.
<code>Asteroidi.cs / Bomba.cs</code>	Gli ostacoli che fanno perdere vite.

Esplosione.cs / Coriandoli.cs / PianetaAmico.cs	Effetti speciali (esplosione, festa, pianeta finale).
InterfacciaGioco.cs	L'HUD: contatori, barra, pulsanti e schermate di vittoria/game over.

Schema mentale. `Avvio` accende tutto → `GestoreGioco` decide le regole → `CaricatoreLivelli` costruisce il livello usando i disegni di `FabbricaImmagini` → `Astro` e gli altri oggetti interagiscono → `InterfacciaGioco` mostra cosa succede.

3. L'avvio: come parte il gioco

3.1 — Avvio.cs

Questo file usa l'attributo `[RuntimeInitializeOnLoadMethod]` : vuol dire che il metodo `Inizia()` viene eseguito **da solo** appena la scena è caricata, senza bisogno di trascinare niente nell'editor. I passi sono:

1. Cancella eventuali telecamere e luci di default già presenti nella scena.
2. Crea una telecamera *ortografica* (vista 2D) con sfondo viola scuro, così sembra lo spazio.
3. Nasconde il cursore del mouse (`Cursor.visible = false`): nel gioco è Astro a fare da puntatore.
4. Crea due oggetti fondamentali: `GestoreGioco` e `InterfacciaGioco` .

3.2 — Impostazioni.cs

È una `static class` che contiene solo costanti: le vite (`VITE`), i secondi per livello (`TEMPO_LIVELLO`), i raggi di presa di Astro, i punti guadagnati/persi e il numero di caramelle del primo livello. L'idea è semplice e molto utile: **tutti i numeri da regolare stanno in un unico file**. Se il gioco è troppo difficile o troppo facile, si cambia un valore qui e tutto il resto si adatta.

Perché è importante per l'esame: centralizzare i parametri mostra una buona pratica di programmazione (evitare i "numeri magici" sparsi nel codice) e rende immediato adattare il gioco al paziente.

4. Il cervello del gioco: GestoreGioco.cs

È la classe centrale. Tiene tutto lo **stato della partita**: livello corrente, caramelle raccolte, punteggio, vite, tempo rimasto, se la missione è completata, se la partita è finita, ecc. Gli altri script lo trovano facilmente grazie al riferimento statico `GestoreGioco.Istanza` (un singleton: una sola copia raggiungibile da tutti).

4.1 — Le tre fasi di ogni livello

Ogni livello si gioca sempre in 3 fasi, gestite da questo file:

1. **Fase 1 — Raccogli le caramelle:** bisogna prenderle tutte.
2. **Fase 2 — Prendi la chiave:** quando le caramelle finiscono, appare una chiave al centro.
3. **Fase 3 — Porta la chiave alla porta:** raggiungere la porta con la chiave completa il livello.

4.2 — Come comunicano gli oggetti col Gestore

Gli altri script non decidono nulla da soli: **avvisano** il Gestore con dei metodi `Segnala...()` , e il Gestore decide cosa fare. Questo tiene le regole tutte in un posto.

Metodo chiamato	Chi lo chiama	Cosa fa il Gestore
<code>SegnalaCaramellaRaccolta()</code>	Caramella	+1 caramella, +punti; se sono finite crea chiave e porta (o vince).
<code>SegnalaChiaveRaccolta()</code>	Chiave	Segna che la chiave è in mano ad Astro.
<code>SegnalaPortaRaggiunta()</code>	Porta	Se ha la chiave: missione completata + festa.
<code>SegnalaAsteroideToccato()</code>	Astro	Toglie punti e una vita (con pausa anti-danni ripetuti).
<code>SegnalaBombaColpita()</code>	Bomba	Toglie una vita.

4.3 — Vite, tempo e periodo "PRONTI"

- **Vite:** ogni colpo (asteroide, bomba o tempo scaduto) toglie una vita; a zero vite è Game Over.
- **Cooldown danno:** dopo un colpo c'è 1 secondo di immunità, così non si perdono tutte le vite restando "incollati" a un asteroide.
- **Tempo:** ogni livello ha un timer; se scade si perde una vita e il timer riparte (finché ci sono vite).
- **Periodo "PRONTI...":** all'inizio di ogni livello c'è un breve istante in cui le bombe non fanno male, per dare il tempo di orientarsi (proprietà `BombeAttive`).

4.4 — Cambio livello e vittoria finale

Il Gestore sa caricare un livello (`CaricaLivello`), passare al successivo (`ProssimoLivello`), ripetere quello corrente o ricominciare da capo. Quando si finiscono le caramelle dell'**ultimo** livello scatta `AttivaVittoriaFinale()` : parte la schermata di congratulazioni con coriandoli e pianeta amico.

5. I livelli: descrizione e costruzione

5.1 — DefinizioneLivelli.cs (la "ricetta" dei livelli)

Qui i livelli sono solo **dati**: per ognuno si dice il titolo, l'obiettivo e dove mettere caramelle, asteroidi e bombe. Vengono usate piccole classi di supporto — `DatiLivello` , `DatiCaramella` , `DatiAsteroide` — che servono come "schede" da riempire.

Livello	Titolo	Cosa contiene
1	Primo Volo	Spazio aperto, nessun ostacolo, 10 caramelle messe a caso.
2	Tra gli Asteroidi	5 barriere-asteroide + 10 caramelle in posizioni scelte a mano.
3	Campo Minato	Stessi asteroidi del livello 2 + 4 bombe + 10 caramelle.

Aggiungere un livello è facile: basta aggiungere un `case` nello `switch` e scrivere un nuovo metodo `CostruisciLivelloN()` , poi aumentare la costante `Conteggio` . Questa separazione tra "cosa c'è in un livello" e "come viene disegnato" è una scelta di progettazione importante.

5.2 — CaricatoreLivelli.cs (il "muratore")

Prende un `DatiLivello` e costruisce davvero gli oggetti in scena. Il metodo `Carica()` :

1. Chiama `Pulisci()` per cancellare il livello precedente (tutto è figlio di un unico oggetto "Missione").
2. Crea lo sfondo: 120 stelline e 4 pianeti decorativi negli angoli.
3. Crea Astro al centro.
4. Crea asteroidi, bombe e caramelle leggendo la descrizione del livello.

Per le caramelle "a caso" (livello 1) c'è `PosizioneCaramellaCasuale()` che prova fino a 30 volte a trovare un punto valido: non troppo vicino al centro (dove parte Astro) e non dentro un asteroide. La chiave e la porta **non** vengono create subito: nascono dopo, quando il Gestore segnala che le caramelle sono finite (`GeneraChiave()` e `GeneraPorta()`). Il caricatore contiene anche gli effetti speciali (`GeneraEsplosione` , `GeneraCoriandoli` , `GeneraPianetaAmico`).

5.3 — FabbricaImmagini.cs (il "disegnatore")

Questo file disegna **ogni sprite del gioco pixel per pixel** su una griglia 16×16. Per ogni oggetto (Astro, caramella, chiave, porta, bomba, asteroide, pianeti, stelle, esplosione) c'è un metodo che riempie un array di colori e poi lo "cuoce" in una `Texture2D` → `Sprite` con il metodo `Cuoci()` . Usando `FilterMode.Point` la grafica resta nitida "a quadretti" (stile pixel art).

Perché disegnare da codice? Così il progetto non dipende da file immagine esterni: è tutto autocontenuto, riproducibile e facile da modificare cambiando qualche pixel. È anche una bella dimostrazione di manipolazione di array e di colori.

6. Il personaggio: Astro.cs

Astro è l'alieno verde controllato dal giocatore. La logica, eseguita ogni frame in `Update()`, è:

1. **Segue il mouse:** converte la posizione del mouse dallo schermo al mondo di gioco (`ScreenToWorldPoint`) e ci si sposta sopra.
2. **Calcola la velocità:** dalla differenza di posizione tra un frame e l'altro; serve per inclinare lo sprite nella direzione del movimento e dargli vita.
3. **Animazioni:** "respiro" continuo, piccolo saltino quando raccoglie una caramella, lampeggio rosso quando prende un colpo.
4. **Controlla i contatti** (solo se il gioco è "vivo"): caramelle, asteroidi, bombe, chiave, porta.

6.1 — Come Astro "tocca" gli oggetti

Una scelta interessante: **non viene usata la fisica di Unity**. I contatti sono calcolati a mano, in modo semplice e prevedibile:

- **Caramelle, chiave, porta, bombe:** si misura la *distanza* tra Astro e l'oggetto; se è sotto un certo *raggio* (preso da `Impostazioni`), conta come tocco.
- **Asteroidi:** sono rettangoli, quindi si controlla se il punto di Astro è *dentro il rettangolo* (`Rect.Contains`).

Raggi generosi = riabilitazione. I raggi di presa sono volutamente ampi (es. chiave e porta 1.8) così non serve precisione millimetrica: il movimento resta un esercizio, non una fonte di frustrazione.

7. Gli oggetti del gioco

7.1 — Caramella.cs

Galleggia con un movimento sinusoidale (ogni caramella con una "fase" diversa, così non si muovono tutte uguali), ruota leggermente e pulsa come se brillasse. Quando Astro le si avvicina, il metodo `Raccogli()` avvisa il Gestore, fa fare il saltino ad Astro e si distrugge. Tutte le caramelle attive sono tenute in una lista statica `Caramella.Active`, così Astro può controllarle velocemente.

7.2 — Chiave.cs

Appare al centro dopo che le caramelle sono finite. Ha due comportamenti:

- **Prima di essere presa:** ondeggia ferma; se Astro entra nel *raggio magnete* (3.5) la chiave gli va incontro (effetto calamita) con velocità crescente; se entra nel *raggio di presa* (1.8) viene presa.
- **Dopo essere presa:** segue Astro con un piccolo ritardo, come un oggetto al traino.

Il **magnetismo** è un altro aiuto pensato per la riabilitazione: avvicinarsi "abbastanza" basta, la chiave fa il resto.

7.3 — Porta.cs

È il traguardo. Per dare un po' di sfida si **sposta da sola** ogni qualche secondo (`PORTA_SECONDI_FERMA`), muovendosi in modo fluido con `SmoothStep` verso una posizione casuale valida (lontana dagli ostacoli e dalle bombe). Si apre **solo** se Astro la tocca portando la chiave (metodo `Sblocca()`).

7.4 — Asteroide.cs e Bomba.cs

- **Asteroidi:** un rettangolo "barriera". Memorizza il proprio `Rect` e offre il metodo `Contiene()`; tutti gli asteroidi stanno nella lista `Asteroide.Tutti`.

- **Bomba:** pulsa di rosso con un alone per farsi notare e ha una piccola "zona di pericolo" attorno. Se Astro la tocca, `Detona()` crea un'esplosione e fa perdere una vita.

7.5 — Effetti speciali

- **Esplosione.cs:** anello luminoso che cresce e svanisce, poi si autodistrugge.
- **Coriandoli.cs:** pioggia di coriandoli colorati che cadono con gravità leggera, per festeggiare.
- **PianetaAmico.cs:** pianeta sorridente che fa "pop" e poi ondeggia a fine missione.

8. L'interfaccia: InterfacciaGioco.cs

Disegna l'HUD usando `OnGUI()`, l'API immediata di Unity: si scrive il disegno riga per riga, senza dover costruire Canvas o prefab. Ogni frame mostra:

Posizione	Contenuto
In alto a sinistra	Caramelle raccolte, punteggio, vite a forma di cuori (♥/♡).
In alto al centro	Titolo del livello e "Livello X di 3".
In alto a destra	Barra "Energia Astro" (% caramelle) + tempo (rosso e lampeggiante sotto i 10s).
In basso a sinistra	Obiettivo del momento ("Raccogli...", "Prendi la chiave", "Porta la chiave...").
In basso a destra	Pulsanti MENU e RIAVVIA.
Al centro	Banner "PRONTI...", pannelli di Missione Completata, Game Over e Vittoria Finale.

Gli stili grafici (font, colori, sfondi delle barre) vengono creati **una volta sola** in `CostruisciStili()` per efficienza. C'è anche un flash rosso su tutto lo schermo quando si prende un colpo, per dare un riscontro visivo immediato (utile come *feedback*). Il pannello di vittoria finale mostra i crediti dell'esame (autore, materia, docente).

9. Il flusso completo di una partita

1. Premo Play → `Avvio` crea telecamera, Gestore e Interfaccia.
 2. Il Gestore carica il Livello 1 → il Caricatore costruisce sfondo, Astro e caramelle.
 3. Breve "PRONTI..." → muovo il mouse, Astro raccoglie le caramelle.
 4. Finite le caramelle → appaiono chiave e porta.
 5. Prendo la chiave (anche grazie al magnetismo) e la porto alla porta.
 6. Missione completata → festa → pulsante "Prossimo Livello".
 7. Livelli 2 e 3 aggiungono asteroidi e bombe (evitare = perdere vite).
 8. Finito il Livello 3 → schermata di Vittoria Finale con i crediti.
- In qualsiasi momento: vite a zero o tempo scaduto troppe volte → Game Over (con possibilità di riprovare).

10. Riepilogo delle scelte di progettazione

- **Tutto da codice:** nessuna immagine o configurazione esterna, progetto autocontenuto e riproducibile.
- **Parametri centralizzati** in `Impostazioni.cs`: facile bilanciare e adattare al paziente.
- **Separazione dei ruoli:** chi descrive i livelli, chi li costruisce, chi li disegna, chi decide le regole e chi mostra l'interfaccia sono file diversi.
- **Comunicazione tramite il Gestore:** gli oggetti "segnalano", il Gestore decide → regole in un solo posto.
- **Contatti calcolati a mano** (distanze e rettangoli) invece della fisica: semplice, prevedibile, regolabile.
- **Aiuti per la riabilitazione:** raggi generosi, magnetismo, periodo "PRONTI", vite e feedback chiari.